BUGPILOT

Application Security Assessment
and Hardening Report

Report detail	Description
Project	BugPilot – AI-Powered Bug Tracking System
Technology stack	React.js + Vite, FastAPI, PostgreSQL 16
Assessment type	Preliminary application security review and hardening plan
Prepared as	Cybersecurity Engineering Review
Date	30 October 2026
Status	Preliminary; source-code audit and penetration testing not yet completed

Purpose: Identify common web application security risks, explain realistic attack scenarios, and provide practical remediation guidance while preserving BugPilot's existing architecture.

Handling: Do not include real passwords, API keys, tokens, or production user data in copies shared for review.

1. Executive Summary

BugPilot is a web-based bug tracking application developed using React.js for the frontend, FastAPI for the backend, and PostgreSQL for persistent data storage. A working application is not necessarily a secure application.

This report covers Cross-Site Scripting (XSS), insecure authentication token storage, hardcoded secrets, vulnerable dependencies, CORS, SQL injection, input validation, authorization, security headers, HTTPS, and error handling.

Assessment limitation: These are potential risks and recommended controls. They have not all been confirmed through source-code inspection or penetration testing.

2. Application Architecture

2.1 Frontend – React.js + Vite

Displays dashboard information, creates and displays bugs, searches and filters records, updates status and assignee, and communicates with the backend API.

2.2 Backend – FastAPI

Handles REST API requests, validates incoming data, processes bug operations, communicates with PostgreSQL, and returns API responses.

2.3 Database – PostgreSQL 16

Stores bug records, status, priority, descriptions, and assignee information, preserving records across application restarts.

2.4 Recommended security architecture

React Frontend → **HTTPS** → **FastAPI Backend** → **SQLAlchemy** → **PostgreSQL.** Authentication, authorization, validation, and business rules must be enforced by the backend.

3. Vulnerability 1 – Cross-Site Scripting (XSS)

Severity: High potential impact. **Category:** Injection / Client-side security.

XSS occurs when an attacker injects malicious JavaScript or HTML into a web application and it executes in another user's browser. In BugPilot, user-submitted bug titles and descriptions must be displayed safely.

3.1 Example attack

An attacker submits this bug description:

```
<script>
  alert("Your account has been compromised!");
</script>
```

If rendered as executable HTML, the script may execute when another user opens the bug. Consequences may include unauthorized browser actions, access to available session information, content manipulation, or phishing.

3.2 Vulnerable React implementation

```
function BugDetails({ bug }) {
  return (
    <div dangerouslySetInnerHTML={{ __html: bug.description }} />
  );
}
```

The `dangerouslySetInnerHTML` property bypasses React's normal text escaping.

3.3 Recommended implementation

```
function BugDetails({ bug }) {
  return <div>{bug.description}</div>;
}
```

React automatically escapes ordinary JSX text interpolation. Avoid unnecessary HTML rendering. If rich text is required, use a maintained sanitizer with a restrictive allowlist and review URL handling.

3.4 Additional controls and verification

• Avoid unsafe DOM manipulation and unnecessary HTML rendering.

• Validate user input on the backend.

• Sanitize HTML if rich-text functionality is genuinely required.

• Apply a suitable Content Security Policy (CSP).

• Test script-like input in titles and descriptions.

Test payload: <img src=x onerror=alert('XSS')>. Expected result: inert text or safely sanitized content; no script executes.

4. Vulnerability 2 – Insecure Authentication Token Storage

Severity: High potential impact. **Category:** Authentication and session management.

An authentication token proves that a user has authenticated. The frontend may send it with subsequent API requests.

4.1 Insecure implementation example

```
localStorage.setItem("token", accessToken);
const token = localStorage.getItem("token");
```

JavaScript executing in the application's origin can access localStorage. If XSS is exploited, an attacker may be able to retrieve the token.

4.2 Recommended approach

For a browser application, consider server-managed sessions using appropriately configured cookies.

Cookie property	Purpose
HttpOnly	Prevents JavaScript from reading the cookie.
Secure	Restricts cookie transmission to HTTPS.
SameSite	Controls cross-site cookie sending.
Path	Limits the cookie's applicable URL paths.

```
response.set_cookie(
    key="session", value=session_token,
    httponly=True, secure=True,
    samesite="lax", path="/"
)
```

This assumes production HTTPS and a properly designed session lifecycle. Cookie-based authentication also requires CSRF protection for state-changing requests.

• Implement CSRF protection for cookie-authenticated writes.

• Set session expiration and invalidate sessions on logout.

• Never expose signing keys in frontend code.

• Avoid storing sensitive credentials in localStorage.

Implementation note: The discussed BugPilot implementation does not yet include a completed authentication system. Token storage must be inspected before identifying an actual vulnerability.

5. Vulnerability 3 – Hardcoded Secrets and Credential Exposure

Severity: Critical potential impact. **Category:** Sensitive data exposure.

Hardcoded secrets are credentials written directly into source code, such as database passwords, API keys, JWT signing secrets, and third-party credentials.

5.1 Vulnerable example

```
const API_KEY = "my-secret-api-key";
```

Frontend JavaScript is delivered to the browser. A secret included in the frontend bundle should be considered publicly accessible.

5.2 BugPilot configuration

```
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@localhost:5432/bugpilot_db
```

Keep actual credentials in a backend environment file and do not commit that file to Git.

5.3 Recommended implementation

```
import os
DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL is not configured")
```

Add to `.gitignore`:

```
.env
.venv/
node_modules/
__pycache__/
*.pyc
```

Maintain an `.env.example` with placeholders:

```
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@localhost:5432/bugpilot_db
```

5.4 Vite environment variables

Vite exposes variables prefixed with `VITE_` to client-side code. A public API base URL may be configured there, but private credentials must never be included.

```
VITE_API_BASE_URL=http://localhost:8000/api
VITE_DATABASE_PASSWORD=mysecretpassword  # NEVER DO THIS
```

5.5 Verification

Search for `password`, `secret`, `api_key`, `token`, and `DATABASE_URL`. Review Git history too. Removing a secret from the latest version does not remove it from previous commits. Rotate any exposed real secret.

6. Vulnerability 4 – Outdated and Vulnerable Dependencies

Severity: Depends on the vulnerability. **Category:** Software supply chain security.

Third-party packages may contain publicly known vulnerabilities. BugPilot's frontend dependencies include React, Vite, and other npm packages. Backend dependencies include FastAPI, SQLAlchemy, Psycopg, and Pydantic.

6.1 Frontend audit

```
npm audit
npm outdated
npm audit --json > npm-audit-report.json
```

6.2 Backend audit

```
cd backend
.\.venv\Scripts\Activate.ps1
python -m pip install pip-audit
pip-audit
```

6.3 Remediation

• Identify affected packages and vulnerable version ranges.

• Check patched versions and release notes.

• Upgrade dependencies in a controlled manner.

• Run the application and regression tests.

• Run the audit again and record the results.

Avoid blindly running `npm audit fix --force`; it may introduce breaking changes.

7. Vulnerability 5 – Insecure CORS Configuration

Severity: Medium to High, depending on configuration. **Category:** Browser security policy.

CORS determines which browser origins are permitted to access an API response. BugPilot currently allows local React development origins.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://127.0.0.1:5173",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

In production, allowing arbitrary origins can expose browser-accessible API responses to untrusted websites. CORS is not authentication: non-browser clients can call an API regardless of browser CORS restrictions.

7.1 Recommended production configuration

```
ALLOWED_ORIGINS = ["https://bugpilot.example.com"]

app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Content-Type", "Authorization", "X-CSRF-Token"],
)
```

Replace the example domain with the actual production frontend domain. Keep development and production configurations separate.

8. Vulnerability 6 – SQL Injection and Database Security

Severity: Critical potential impact. **Category:** Injection / Database security.

SQL injection occurs when an attacker manipulates a database query by providing specially crafted input.

8.1 Vulnerable example

```
query = (
    "SELECT * FROM bugs WHERE title = '"
    + user_input
    + "'"
)
```

Direct concatenation of user input into SQL statements can allow query manipulation.

8.2 Recommended SQLAlchemy implementation

```
from sqlalchemy import select
statement = select(Bug).where(Bug.title == user_input)
results = db.scalars(statement).all()
```

SQLAlchemy's expression API binds values as data rather than treating them as SQL syntax. Avoid raw SQL string concatenation with untrusted values.

8.3 Additional database protections

• Use a dedicated application database account.

• Grant only necessary privileges; do not use a PostgreSQL superuser for application operations.

• Keep credentials outside source control.

• Restrict network access to PostgreSQL.

• Maintain backups and test recovery.

• Use versioned database migrations.

9. Vulnerability 7 – Missing Input Validation and Authorization

Severity: High potential impact. **Category:** Access control and input validation.

Attackers may submit unexpected or excessively large values. The backend must reject invalid values regardless of frontend validation.

```
{
  "title": "",
  "description": "Invalid or excessively large content",
  "priority": "Unknown"
}
```

9.1 Recommended Pydantic validation

```
from pydantic import BaseModel, Field
from typing import Literal

class BugCreate(BaseModel):
    title: str = Field(min_length=1, max_length=150)
    description: str = Field(min_length=1, max_length=5000)
    priority: Literal[
        "Critical", "High", "Medium", "Low"
    ] = "Medium"
```

9.2 Authorization example

Suppose User A creates a bug and User B attempts to access or update it. The backend must verify whether User B has permission. Hiding an Update button in React is not sufficient. Authorization must be checked on every relevant backend request.

10. Vulnerability 8 – Insecure Error Handling and Security Headers

10.1 Error handling

Avoid returning internal implementation details to users. Database exceptions may reveal table names, SQL details, file paths, or internal configuration.

```
except Exception:
    logger.exception("Unexpected application error")
    raise HTTPException(
        status_code=500,
        detail="An internal server error occurred"
    )
```

Log errors securely without recording passwords, tokens, or sensitive user content.

10.2 Security headers

• Content-Security-Policy (CSP).

• X-Content-Type-Options.

• Referrer-Policy.

• Strict-Transport-Security (HSTS).

• Frame protection through CSP frame-ancestors.

Apply headers at the application or reverse-proxy layer and test the policy against actual frontend assets and integrations.

11. Security Testing Plan

Test ID	Security test	Expected result
SEC01	Submit script-like content as bug description	Script remains inert
SEC02	Inspect browser storage	No sensitive authentication token in localStorage
SEC03	Inspect frontend build	No private API or database credentials
SEC04	Run npm audit	Findings reviewed and remediated where applicable
SEC05	Run pip-audit	Findings reviewed and remediated where applicable
SEC06	Send invalid priority to API	Backend returns validation error
SEC07	Send unauthorized API request	Request rejected

Test ID	Security test	Expected result
SEC08	Access API from untrusted browser origin	Browser access restricted
SEC09	Submit SQL injection-like input	Input handled as data
SEC10	Trigger unexpected backend error	No sensitive internal details exposed

These tests must be executed against the actual implementation before declaring them passed.

12. Recommended Security Hardening Roadmap

Phase	Task	Priority
1	Review React components for XSS	High
2	Inspect browser token and session storage	High
3	Remove and protect exposed secrets	Critical
4	Audit npm and Python dependencies	High
5	Harden CORS and security headers	High
6	Review SQLAlchemy queries and database privileges	High
7	Implement authentication and authorization	Critical before multi-user deployment
8	Add automated security tests	High
9	Configure HTTPS and production environment	Critical before public deployment

13. Conclusion

Security hardening should be an ongoing engineering activity rather than a one-time task. For BugPilot, the immediate priority is to review the existing frontend and backend code, protect database credentials, ensure safe rendering of user-generated content, and audit third-party dependencies.

Before production deployment, the application should also implement proper authentication, authorization, secure session handling, HTTPS, logging, backups, and automated security testing.

Final recommendation: Complete the security review and remediation process before exposing BugPilot to public users or storing sensitive organizational information.

Assessment status: Preliminary security assessment and remediation plan. No claim of a completed penetration test or independently verified security certification is made.